SKILL - 14

Name: k.vaishali

Rollno: 2520030117

Section: 8

Aim

To parse the append operator, open files in append mode, redirect output streams, preserve existing data, validate append operations, and test different append scenarios.

Objective

- To parse the append operator.
- To open files in append mode.
- To redirect output streams.
- To preserve existing file data.
- To validate append operations.
- To test different append scenarios.
- To parse stderr redirection.
- To redirect standard error streams.
- To capture error output.

Problem Statement

Design and implement a shell-style redirection mechanism that recognizes the append operator and writes new output at the end of an existing file without deleting its previous contents. The program should also recognize stderr redirection, redirect error messages to a file, verify the resulting file contents, handle failed commands, and test both successful and error scenarios.

Theory

1. Append Operator

The append operator >> sends standard output to a file while preserving the data that is already stored in that file. The file is opened with append access so new output is written after the existing content.

2. Append Mode

In C, a file can be opened in append mode using O_APPEND together with O_WRONLY and O_CREAT when a file should be created if it does not already exist. The file offset is positioned for writing at the end of the file.

3. Stream Redirection

dup2() can replace standard output or standard error with an open file descriptor. Once redirected, data written to

that standard stream is sent to the selected file.

4. Preserving Existing Data

Append redirection must not truncate the destination file. Existing lines remain unchanged, and new output is added after them.

5. stderr Redirection

Standard error is represented by file descriptor 2. Redirecting stderr to a file allows error messages from unsuccessful commands to be captured separately from normal output.

6. Error Handling

A command may fail because an executable does not exist or because another execution error occurs. The child process can report the error through stderr, while the parent waits and checks the termination status.

7. File Verification

After a successful append or error redirection, the destination file can be opened and displayed to verify that the expected data was preserved or captured.

8. Append Scenarios

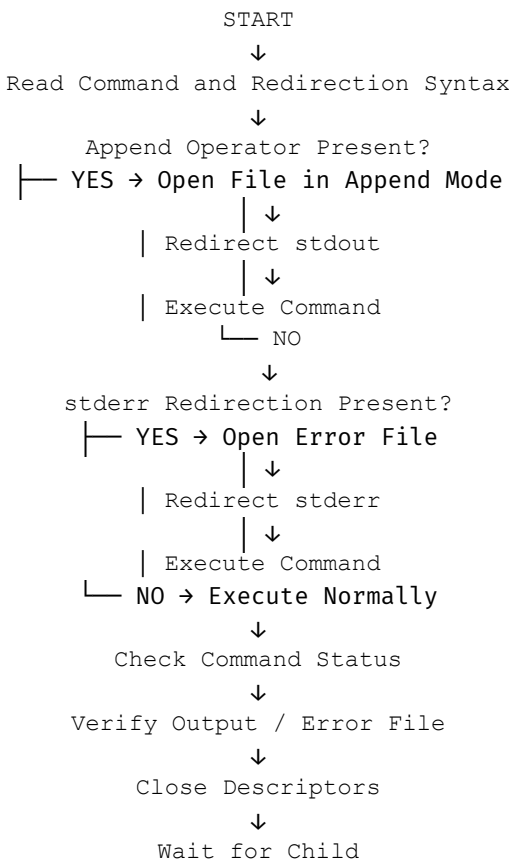
Appending can be tested with an existing file containing initial text, followed by one or more commands whose output is added to the same file. The original data should remain visible.

9. Error Scenarios

stderr redirection can be tested with a valid command and an invalid command. The valid command should not produce an error message, while a failed command should write its diagnostic output to the error file.

10. Control Flow

The complete control flow is kept on this page so that the flowchart is not divided across pages.



↓
STOP

11. Algorithm

1. Start the program.
2. Create an initial output file containing sample data.
3. Prepare the command for append redirection.
4. Open the destination file using append mode.
5. Redirect standard output to the opened file.
6. Create a child process and execute a valid command.
7. Close unused file descriptors in the child and parent.
8. Wait for the command to finish.
9. Open the error-output file for writing.
10. Redirect standard error to the error file.
11. Create a child process and execute an invalid command.
12. Capture the generated error message in the error file.
13. Wait for the failed child process.
14. Read and display the contents of the output and error files.
15. Verify that previous data was preserved and errors were captured.
16. Stop the program.

12. Data Used

Character arrays store command names, file names, and file contents. File descriptors store the opened append and error files. Process IDs and status values are used to manage and synchronize child processes.

13. Operations Used

- open() with append flags for file handling.
- dup2() for standard-output and standard-error redirection.
- fork() for creating child processes.
- execlp() for executing commands.
- close() for descriptor cleanup.
- waitpid() for process synchronization.
- read() and write() for file verification.

14. Redirection Structure

The required stream connections are:

```
Command stdout ──>> output.txt
Command stderr ──> errors.txt
```

15. Program

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
```

```
#include <sys/wait.h>
#include <sys/stat.h>
#include <string.h>
#define SIZE 512
void show_file(const char *name) {
 char buffer[SIZE];
 int fd = open(name, O_RDONLY);

 if (fd < 0) {
 perror(name);
 return;
 }

 printf("\n--- %s ---\n", name);

 ssize_t n;
 while ((n = read(fd, buffer, SIZE - 1)) > 0) {
buffer[n] = '\0';
 printf("%s", buffer);
 }

 close(fd);
 }

int main() {
 int fd;
 pid_t pid;
 int status;

 printf("Skill - 14: Append and stderr Redirection\n");
printf("-----------------------------------------\n");

 /* Initial file content */
 fd = open("append.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

 if (fd < 0) {
 perror("append.txt");
 return 1;
 }

 write(fd, "Original data preserved.\n", 25);
close(fd);

 printf("Initial file created.\n");

 /* Append test */
 pid = fork();

 if (pid == 0) {
 fd = open("append.txt",
 O_WRONLY | O_CREAT | O_APPEND, 0644);

 if (fd < 0) {
 perror("open append");
 exit(1);
 }

 dup2(fd, STDOUT_FILENO);
 close(fd);

 execlp("echo", "echo",
 "New data appended successfully.", NULL);

 perror("echo");
 exit(1);
 }

 waitpid(pid, &status, 0);

 printf("Append operation completed.\n");

 /* stderr redirection test */
 pid = fork();

 if (pid == 0) {
```

```
fd = open("errors.txt",
O_WRONLY | O_CREAT | O_TRUNC, 0644);

if (fd < 0) {
perror("open errors");
exit(1);
}

dup2(fd, STDERR_FILENO);
close(fd);

execlp("nonexistent_command",
"nonexistent_command", NULL);

perror("nonexistent_command");
exit(1);
}

waitpid(pid, &status, 0);

printf("stderr redirection test completed.\n");

show_file("append.txt");
show_file("errors.txt");

if (WIFEXITED(status) && WEXITSTATUS(status) != 0)
printf("\nInvalid command failure handled correctly.\n");

printf("Redirection tests completed successfully.\n");

return 0;
}
```

16. Program Explanation

16.1 Initial File

The program first creates append.txt with original content. This provides existing data that can be checked after the append operation.

16.2 Append Redirection

The child opens append.txt using O_APPEND. dup2() connects standard output to the file, and echo writes new content without removing the original line.

16.3 stderr Redirection

The second child opens errors.txt and redirects standard error using dup2(). A deliberately invalid command is executed so that the error message is captured in the file.

16.4 File Verification

The show_file() function opens each resulting file, reads its contents, and displays them. This verifies that the original data remains and that the error output was captured.

16.5 Failure Handling

The invalid command fails as expected. The parent waits for the child and reports that the failure was handled correctly.

17. Sample Output

```
Skill - 14: Append and stderr Redirection
-----------------------------------------
Initial file created.
Append operation completed.
stderr redirection test completed.
--- append.txt ---
```

```
Original data preserved.
New data appended successfully.

--- errors.txt ---
nonexistent_command: command not found

 Invalid command failure handled correctly.
Redirection tests completed successfully.
```

18. Output Screenshot

Paste the screenshot of the program output in the space provided below.



19. Testing

- Tested creation of the initial file.
- Tested append mode with an existing file.
- Verified that original data was preserved.
- Tested standard-output redirection.
- Tested standard-error redirection.
- Verified that invalid-command errors were captured in the error file.
- Verified that child-process failures were handled and synchronized.

20. Test Cases

Test Case	Input / Scenario Expected Result

1	Existing append.txt Original contents remain unchanged.
2	echo text >> append.txt New text is added after existing data.
3	Valid command with stderr redirection No unexpected error is written.
4	Invalid command > stderr file Error message is captured in the error file.
5	Repeated append Each new output is added without truncation.
6	Child process failure Parent detects completion and handles failure.

21. Observation

- The existing contents of append.txt were preserved.
- New output was added at the end of the file.
- Standard output was redirected successfully.
- Standard error was redirected to a separate file.
- The invalid command generated error output as expected.
- The error file contained the captured diagnostic message.
- The parent process waited for the child process before verification.

22. Result

Append and stderr redirection were successfully implemented. The append operation preserved existing file contents while adding new output, and standard error from a failed command was captured in a separate file. The resulting files were verified and child processes were synchronized correctly.

23. Conclusion

The practical demonstrated how an operating system manages standard streams and file descriptors for shell redirection. Append mode allows new data to be added without losing existing contents, while stderr redirection provides a controlled way to capture and verify error messages from failed commands.